

Android Helios

Informe Técnico del Proyecto

App Android: Linterna controlada por sacudida

Acelerómetro + CameraManager + Gráfico en tiempo real + Foreground Service

Autor: Facundo Arito
Materia: Computación Aplicada a la Ingeniería
Repositorio: github.com/wopassord/Android_Helios
Fecha: Junio de 2026

Contents

1	Introducción	2
2	Estructura del Proyecto	2
3	La Linterna: CameraManager	3
3.1	API utilizada	3
3.2	Búsqueda del ID de cámara con flash	3
3.3	Toggle de la linterna	3
3.4	Sincronización de estado con TorchCallback	4
3.5	Actualización del indicador visual	4
4	El Acelerómetro: SensorManager	5
4.1	API utilizada	5
4.2	Ciclo de vida: registro y desregistro del listener	5
5	Detección de Sacudida	6
5.1	Cálculo de la magnitud	6
5.2	Algoritmo de detección	6
5.3	Constantes y umbral configurable	7
6	Gráfico en Tiempo Real: AccelerometerGraphView	8
6.1	Vista personalizada con Canvas	8
6.2	Buffer circular	8
6.3	Inicialización de los pinceles	9
6.4	Método onDraw	9
7	Ejecución en Segundo Plano: TorchService	10
7.1	Motivación	10
7.2	Arquitectura del servicio	10
7.3	Traspaso de control entre Activity y Service	11
7.4	Toggle en la interfaz	12
7.5	Declaración en el Manifest	12
8	Conclusiones	13

1. Introducción

Android Helios es una aplicación Android desarrollada en Java cuyo objetivo principal es **encender o apagar la linterna del teléfono mediante un patrón de sacudida configurable**. La app integra dos módulos de hardware:

- **Acelerómetro:** detecta el movimiento del dispositivo, calcula la magnitud de la aceleración en unidades de gravedad y aplica un algoritmo de detección de sacudidas.
- **Cámara / Flash:** controla la linterna a través de `CameraManager.setTorchMode()`, sin necesidad de abrir un stream de cámara.

La interfaz expone tres controles al usuario: un indicador de estado de la linterna (círculo cromado + texto), un gráfico en tiempo real de la aceleración medida, y un deslizador de sensibilidad. Un toggle en la esquina superior derecha permite que la detección continúe activa incluso cuando el usuario sale de la aplicación.

2. Estructura del Proyecto

El proyecto sigue la estructura estándar de Android Studio con Gradle (Kotlin DSL). Los archivos programados son:

Archivo	Rol
MainActivity.java	Activity única. Coordina el acelerómetro, la linterna, el gráfico en tiempo real y toda la lógica de UI.
AccelerometerGraphView.java	Vista personalizada (<code>extends View</code>) que dibuja el gráfico de g-force con Canvas de Android.
TorchService.java	Foreground Service. Replica la detección de sacudidas y el toggle de la linterna cuando la app está en segundo plano.
activity_main.xml	Layout principal: indicador circular, gráfico, slider de sensibilidad y toggle de segundo plano.
AndroidManifest.xml	Declara permisos (CAMERA, FOREGROUND_SERVICE) y registra el servicio.
drawable/circle_indicator.xml	Shape drawable oval que sirve de indicador visual del estado de la linterna.

3. La Linterna: CameraManager

3.1. API utilizada

Android expone el control del flash a través de `android.hardware.camera2.CameraManager`. El método `setTorchMode(String cameraId, boolean enabled)`, disponible desde API 23, enciende o apaga el flash sin abrir ningún stream de cámara ni requerir el permiso `CAMERA` de manera obligatoria.

3.2. Búsqueda del ID de cámara con flash

Cada dispositivo puede tener varias cámaras. Antes de usar la linterna hay que identificar cuál tiene flash. Esto se resuelve en `findTorchCameraId()`, llamada una única vez desde `onCreate()`:

```

1 private String findTorchCameraId() {
2     try {
3         for (String id : cameraManager.getCameraIdList()) {
4             Boolean hasFlash = cameraManager
5                 .getCameraCharacteristics(id)
6                 .get(CameraCharacteristics.
7                     FLASH_INFO_AVAILABLE);
8             if (Boolean.TRUE.equals(hasFlash)) return id;
9         }
10    } catch (CameraAccessException e) {
11        Log.e("MainActivity", "Impossible de trouver la camera
12        torch", e);
13    }
14    return null;
15 }
```

Listing 1: Búsqueda del ID de cámara — MainActivity.java

`getCameraIdList()` devuelve un array con los identificadores de todas las cámaras disponibles. Para cada una, se consulta la propiedad `FLASH_INFO_AVAILABLE` de sus características. Se retorna el primer ID cuyo flash esté disponible, o `null` si ninguno lo tiene. Si el resultado es `null`, `onShakeDetected()` no hace nada.

3.3. Toggle de la linterna

El encendido y apagado ocurre en `onShakeDetected()`, disparada por el algoritmo de detección cuando se cumple el patrón de sacudida:

```

1 private void onShakeDetected() {
2     if (torchCameraId == null) return;
3     try {
4         cameraManager.setTorchMode(torchCameraId, !torchOn);
5     } catch (CameraAccessException e) {
6         Log.e("MainActivity", "Impossible de basculer la torche",
7         e);
8     }
9 }
```

Listing 2: Toggle de la linterna — MainActivity.java

Notar que `torchOn` *no* se modifica aquí directamente. La fuente de verdad del estado real del flash es el `TorchCallback` (sección siguiente), lo que evita desincronizaciones si

otra app o el TorchService modifica el flash en paralelo.

3.4. Sincronización de estado con TorchCallback

CameraManager provee un mecanismo de callback para recibir notificaciones cuando el estado del flash cambia, sin importar el origen del cambio:

```

1  private final CameraManager.TorchCallback torchCallback =
2      new CameraManager.TorchCallback() {
3      @Override
4      public void onTorchModeChanged(String cameraId, boolean
5      enabled) {
6          if (cameraId.equals(torchCameraId)) {
7              torchOn = enabled;
8              updateTorchUI();
9          }
10     }
11 };
12 // En onCreate():
13 cameraManager.registerTorchCallback(
14     torchCallback,
15     new Handler(Looper.getMainLooper()));
16
17 // En onDestroy():
18 cameraManager.unregisterTorchCallback(torchCallback);

```

Listing 3: Registro y uso del TorchCallback — MainActivity.java

El Handler con `Looper.getMainLooper()` garantiza que `onTorchModeChanged()` se ejecute en el hilo principal, permitiendo actualizar la UI directamente desde el callback sin necesidad de `runOnUiThread()`.

3.5. Actualización del indicador visual

Cuando el callback dispara `updateTorchUI()`, se modifica el color del View circular (un `GradientDrawable`) y el texto de estado:

```

1  private void updateTorchUI() {
2      GradientDrawable drawable =
3          (GradientDrawable) viewTorchIndicator.getBackground();
4      if (torchOn) {
5          drawable.setColor(0xFFFFFC107); // ambar
6          tvTorchStatus.setText("ALLUMÉE");
7          tvTorchStatus.setTextColor(0xFFFFFC107);
8      } else {
9          drawable.setColor(0xFF9E9E9E); // gris
10         tvTorchStatus.setText("ÉTEINTE");
11         tvTorchStatus.setTextColor(0xFF9E9E9E);
12     }
13 }

```

Listing 4: Actualización del indicador de estado — MainActivity.java

El color del drawable se modifica en tiempo de ejecución mediante `setColor()`: el círculo pasa de gris (#9E9E9E) a ámbar (#FFC107) al encender la linterna, y vuelve a

gris al apagarla. Esta es la única función de la app que modifica elementos de UI, lo que centraliza la lógica de presentación en un único lugar.

4. El Acelerómetro: *SensorManager*

4.1. API utilizada

El sistema de sensores de Android se accede a través de `android.hardware.SensorManager`. Para recibir lecturas del acelerómetro, `MainActivity` implementa la interfaz `SensorEventListener`, que obliga a definir dos métodos: `onSensorChanged()` y `onAccuracyChanged()`.

El acelerómetro se obtiene e inicializa en `onCreate()`:

```

1  sensorManager = (SensorManager) getSystemService(Context.
    SENSOR_SERVICE);
2  if (sensorManager != null) {
3      accelerometer = sensorManager.getDefaultSensor(Sensor.
        TYPE_ACCELEROMETER);
4  }
5  if (accelerometer == null) {
6      Log.e("MainActivity", "Cet appareil n'a pas d'accelerometre");
7  }

```

Listing 5: Inicialización del acelerómetro — `MainActivity.java`

4.2. Ciclo de vida: registro y desregistro del listener

Un aspecto crítico del uso de sensores en Android es gestionar correctamente el ciclo de vida. El listener se registra únicamente cuando la app está en primer plano y se desregistra al salir, lo que ahorra batería y evita lecturas innecesarias:

```

1  @Override
2  protected void onResume() {
3      super.onResume();
4      stopService(new Intent(this, TorchService.class));
5      if (accelerometer != null) {
6          sensorManager.registerListener(
7              this,
8              accelerometer,
9              SensorManager.SENSOR_DELAY_NORMAL);
10     }
11 }
12
13 @Override
14 protected void onPause() {
15     super.onPause();
16     if (accelerometer != null) {
17         sensorManager.unregisterListener(this);
18     }
19     if (backgroundEnabled) {
20         Intent intent = new Intent(this, TorchService.class);
21         intent.putExtra(TorchService.EXTRA_THRESHOLD,
            shakeThreshold);

```

```

22         intent.putExtra(TorchService.EXTRA_TORCH_STATE, torchOn);
23         startForegroundService(intent);
24     }
25 }

```

Listing 6: Gestión del ciclo de vida del sensor — MainActivity.java

SENSOR_DELAY_NORMAL define una cadencia de muestreo de aproximadamente 200 ms entre lecturas. En `onResume()` también se detiene el `TorchService` si estuviese corriendo, para garantizar que en todo momento *solo un* componente escucha el sensor.

5. Detección de Sacudida

5.1. Cálculo de la magnitud

El acelerómetro entrega tres valores: aceleración en los ejes X , Y y Z expresada en m/s^2 . Para detectar una sacudida independientemente de la orientación del teléfono, se calcula la **magnitud del vector de aceleración** normalizada en unidades de gravedad (g-force):

$$g_{\text{force}} = \sqrt{\left(\frac{a_x}{g}\right)^2 + \left(\frac{a_y}{g}\right)^2 + \left(\frac{a_z}{g}\right)^2}$$

donde $g = 9,80665 \text{ m/s}^2$ (`SensorManager.GRAVITY_EARTH`). Con el teléfono en reposo, el vector apunta en la dirección de la gravedad con magnitud $\approx 1g$. Una sacudida produce picos de $2,5g$ – $5g$.

5.2. Algoritmo de detección

La función `onSensorChanged()` se ejecuta con cada nueva lectura del sensor y concentra toda la lógica de detección:

```

1  @Override
2  public void onSensorChanged(SensorEvent event) {
3      float gX = event.values[0] / SensorManager.GRAVITY_EARTH;
4      float gY = event.values[1] / SensorManager.GRAVITY_EARTH;
5      float gZ = event.values[2] / SensorManager.GRAVITY_EARTH;
6      float gForce = (float) Math.sqrt(gX * gX + gY * gY + gZ * gZ);
7
8      graphAccelerometer.addValue(gForce);    // actualiza el grafico
9
10     if (gForce < shakeThreshold) return;    // filtro 1: umbral
11
12     long now = System.currentTimeMillis();
13     if (now - lastShakeTime < SHAKE_SLOP_TIME_MS) return; //
14     // filtro 2
15
16     if (now - lastShakeTime > SHAKE_COUNT_RESET_TIME_MS) {
17         shakeCount = 0;                      // filtro 3: ventana
18         // temporal
19     }

```

```

18
19     lastShakeTime = now;
20     shakeCount++;
21
22     if (shakeCount >= MIN_SHAKE_COUNT) {
23         shakeCount = 0;
24         onShakeDetected();
25     }
26 }

```

Listing 7: Detección de sacudida en tiempo real — MainActivity.java

El algoritmo funciona en cuatro capas de filtrado superpuestas:

1. **Filtro de umbral** (`shakeThreshold`): si `gForce` no supera el umbral, la lectura se descarta. El umbral es ajustado en tiempo real por el slider de sensibilidad (de 4,0 *g* poco sensible a 1,5 *g* muy sensible).
2. **Tiempo mínimo entre sacudidas** (`SHAKE_SLOP_TIME_MS = 500 ms`): evita que un único movimiento brusco cuente como múltiples sacudidas mientras la lectura del sensor sigue superando el umbral.
3. **Ventana temporal de reinicio** (`SHAKE_COUNT_RESET_TIME_MS = 3000 ms`): si transcurren más de 3 segundos entre dos sacudidas válidas, el contador se pone a cero. Esto obliga a que las sacudidas sean consecutivas y no acumuladas a lo largo del tiempo.
4. **Contador mínimo** (`MIN_SHAKE_COUNT = 2`): se necesitan al menos 2 sacudidas válidas y consecutivas para disparar `onShakeDetected()`. Esto previene activaciones accidentales por un único golpe.

5.3. Constantes y umbral configurable

Los parámetros de tiempo son constantes de clase; `shakeThreshold` es un campo mutable para que el slider pueda modificarlo en tiempo real:

```

1 private static final int SHAKE_SLOP_TIME_MS      = 500;
2 private static final int SHAKE_COUNT_RESET_TIME_MS = 3000;
3 private static final int MIN_SHAKE_COUNT         = 2;
4
5 private float shakeThreshold = 2.75f;

```

Listing 8: Constantes del algoritmo de detección — MainActivity.java

La fórmula de mapeo del slider al umbral:

```

1 seekBarSensitivity.setOnSeekBarChangeListener(
2     new SeekBar.OnSeekBarChangeListener() {
3         @Override
4         public void onProgressChanged(SeekBar seekBar,
5             int progress,
6             boolean fromUser) {
7             shakeThreshold = 4.0f - (progress * 0.25f);
8             tvSensitivityValue.setText(progress + " / 10");
9             graphAccelerometer.setThreshold(shakeThreshold);
10    }

```



```

11     @Override public void onStartTrackingTouch(SeekBar seekBar) {}
12     @Override public void onStopTrackingTouch(SeekBar seekBar) {}
13 };

```

Listing 9: Ajuste dinámico del umbral desde el SeekBar — MainActivity.java

La fórmula $4.0 - \text{progress} * 0.25$ mapea el rango $[0, 10]$ del slider al rango de umbral $[4,0\text{ g}, 1,5\text{ g}]$: a mayor nivel de sensibilidad (slider a la derecha), menor es el umbral y más fácil resulta disparar la detección. El valor por defecto es `progress = 5` ($\rightarrow 2,75\text{ g}$).

6. Gráfico en Tiempo Real: AccelerometerGraphView

6.1. Vista personalizada con Canvas

Para visualizar los valores del acelerómetro en tiempo real se implementó `AccelerometerGraphView`, una clase que extiende `android.view.View` y dibuja directamente sobre un `Canvas`.

El gráfico muestra:

- **Línea azul:** magnitud `gForce` en tiempo real.
- **Línea roja punteada:** umbral actual, se desplaza verticalmente al mover el slider.
- **Grilla gris:** líneas horizontales cada 1 g .

6.2. Buffer circular

La vista mantiene un buffer circular de 150 muestras. Al llegar una nueva lectura, sobrescribe la posición más antigua y el puntero `head` avanza en módulo:

```

1  private static final int    BUFFER_SIZE = 150;
2  private static final float Y_MAX      = 5.0f;
3
4  private final float[] values = new float[BUFFER_SIZE];
5  private int    head      = 0;
6  private int    count     = 0;
7  private float  threshold = 2.75f;
8
9  public void addValue(float gForce) {
10      values[head] = gForce;
11      head = (head + 1) % BUFFER_SIZE;
12      if (count < BUFFER_SIZE) count++;
13      postInvalidate();
14  }
15
16  public void setThreshold(float threshold) {
17      this.threshold = threshold;
18      postInvalidate();
19  }

```

Listing 10: Buffer circular y actualización — AccelerometerGraphView.java

`postInvalidate()` es *thread-safe*: puede llamarse desde cualquier hilo (incluyendo el hilo del sensor) y programa un redibujo en el hilo de UI. El eje Y del gráfico cubre de 0 a 5 g (Y_MAX), rango que contiene en todo momento la línea de umbral (entre 1,5 g y 4,0 g).

6.3. Inicialización de los pinceles

Los objetos `Paint` se crean *únicamente en el constructor*, nunca dentro de `onDraw()`. Esta es una regla fundamental en Android: instanciar objetos en `onDraw()` genera basura en cada frame y degrada significativamente el rendimiento.

```

1  public AccelerometerGraphView(Context context, AttributeSet attrs)
2  {
3
4      linePaint = new Paint(Paint.ANTI_ALIAS_FLAG);
5      linePaint.setColor(0xFF42A5F5);           // azul Material Blue 400
6      linePaint.setStrokeWidth(3f);
7      linePaint.setStyle(Paint.Style.STROKE);
8
9      thresholdPaint = new Paint(Paint.ANTI_ALIAS_FLAG);
10     thresholdPaint.setColor(0xFFEF5350); // rojo Material Red 400
11     thresholdPaint.setStrokeWidth(2f);
12     thresholdPaint.setStyle(Paint.Style.STROKE);
13     thresholdPaint.setPathEffect(
14         new DashPathEffect(new float[]{16f, 8f}, 0));
15
16     gridPaint = new Paint();
17     gridPaint.setColor(0x33FFFFFF);           // blanco al 20% de
18     gridPaint.setStrokeWidth(1f);             opacidad
19 }

```

Listing 11: Inicialización de `Paint` en el constructor — `AccelerometerGraphView.java`

El `DashPathEffect` de la línea de umbral recibe un array de dos valores: `[16, 8]` significa segmento de 16 px encendido, 8 px apagado, generando el efecto de línea punteada.

6.4. Método `onDraw`

`onDraw()` se ejecuta cada vez que la vista necesita redibujarse. Dibuja tres capas en orden sobre el `Canvas`:

```

1  @Override
2  protected void onDraw(Canvas canvas) {
3      int w = getWidth();
4      int h = getHeight();
5
6      canvas.drawColor(0xFF1E1E1E);           // fondo oscuro VSCode
7
8      for (int g = 1; g <= 4; g++) {           // grilla cada 1g
9          float y = h * (1f - g / Y_MAX);
10         canvas.drawLine(0, y, w, y, gridPaint);
11     }
12 }

```

```

13     float ty = h * (1f - threshold / Y_MAX); // línea de umbral
14     canvas.drawLine(0, ty, w, ty, thresholdPaint);
15
16     if (count < 2) return;
17
18     path.reset();
19     float xStep = (float) w / (BUFFER_SIZE - 1);
20     for (int i = 0; i < count; i++) {
21         int idx = (head - count + i + BUFFER_SIZE) % BUFFER_SIZE
22         ;
23         float x = i * xStep;
24         float y = h * (1f - Math.min(values[idx], Y_MAX) / Y_MAX
25     );
26         if (i == 0) path.moveTo(x, y);
27         else path.lineTo(x, y);
28     }
29     canvas.drawPath(path, linePaint);
30 }

```

Listing 12: Dibujo del gráfico con Canvas — AccelerometerGraphView.java

La coordenada vertical en píxeles para un valor v se calcula como:

$$y_{\text{px}} = h \cdot \left(1 - \frac{v}{Y_{\text{max}}}\right)$$

El factor $(1 - \cdot)$ invierte el eje porque en el sistema de coordenadas del Canvas de Android el eje Y crece hacia abajo (el origen está en la esquina superior izquierda). El índice `idx` recorre el buffer circular en orden cronológico: del dato más antiguo (izquierda) al más reciente (derecha), produciendo el efecto de scroll hacia la izquierda a medida que llegan nuevas muestras.

7. Ejecución en Segundo Plano: TorchService

7.1. Motivación

El comportamiento predeterminado de Android es suspender el acelerómetro cuando la Activity entra en pausa (al salir de la app). Para mantener la funcionalidad activa en segundo plano, Android exige usar un **Foreground Service**: un servicio que muestra una notificación persistente, informando al usuario que la aplicación sigue activa. Esto impide que el sistema operativo mate el proceso por falta de recursos.

7.2. Arquitectura del servicio

TorchService extiende Service e implementa SensorEventListener, replicando exactamente la misma lógica de detección de sacudidas de MainActivity:

```

1  @Override
2  public int onStartCommand(Intent intent, int flags, int startId) {
3      if (intent != null) {

```

```

4      shakeThreshold = intent.getFloatExtra(EXTRA_THRESHOLD,
5      2.75f);
6      torchOn      = intent.getBooleanExtra(EXTRA_TORCH_STATE,
7      false);
8      }
9      createNotificationChannel();
10     startForeground(NOTIF_ID, buildNotification());
11     if (accelerometer != null) {
12         sensorManager.registerListener(
13             this,
14             accelerometer,
15             SensorManager.SENSOR_DELAY_NORMAL);
16     }
17     return START_STICKY;
18 }

```

Listing 13: Arranque del servicio — TorchService.java

El servicio recibe el umbral de sensibilidad y el estado actual del flash como `Intent extras`, para continuar desde donde lo dejó la Activity. `startForeground()` debe invocarse antes de que transcurran 5 segundos desde el inicio del servicio. `START_STICKY` indica al sistema operativo que, si necesita terminar el proceso, debe reiniciarlo automáticamente.

La notificación persistente se construye con `NotificationCompat`:

```

1 private Notification buildNotification() {
2     Intent intent = new Intent(this, MainActivity.class);
3     PendingIntent pi = PendingIntent.getActivity(
4         this, 0, intent, PendingIntent.FLAG_IMMUTABLE);
5     return new NotificationCompat.Builder(this, CHANNEL_ID)
6         .setContentTitle("Helios actif")
7         .setContentText("Secouez pour allumer/eteindre la
8         torche")
9         .setSmallIcon(R.mipmap.ic_launcher)
10        .setContentIntent(pi)
11        .build();
12 }

```

Listing 14: Construcción de la notificación — TorchService.java

El `PendingIntent` con `FLAG_IMMUTABLE` permite al usuario tocar la notificación para volver a la Activity.

7.3. Traspaso de control entre Activity y Service

El diseño garantiza que en todo momento *solo uno* de los dos componentes escucha el acelerómetro:

Estado de la app	Toggle fondo	Quién escucha
En primer plano	OFF	MainActivity
En primer plano	ON	MainActivity
En segundo plano	OFF	Nadie (sensor inactivo)
En segundo plano	ON	TorchService

Al volver a la app, `onResume()` detiene el servicio *antes* de registrar el listener en la Activity:

```

1  @Override
2  protected void onResume() {
3      super.onResume();
4      stopService(new Intent(this, TorchService.class));
5      if (accelerometer != null) {
6          sensorManager.registerListener(
7              this,
8              accelerometer,
9              SensorManager.SENSOR_DELAY_NORMAL);
10     }
11 }

```

Listing 15: Traspaso de vuelta a la Activity — MainActivity.java

Al salir de la app, `onPause()` desregistra el listener de la Activity y, si el toggle está activo, lanza el servicio:

```

1  @Override
2  protected void onPause() {
3      super.onPause();
4      if (accelerometer != null) {
5          sensorManager.unregisterListener(this);
6      }
7      if (backgroundEnabled) {
8          Intent intent = new Intent(this, TorchService.class);
9          intent.putExtra(TorchService.EXTRA_THRESHOLD,
10             shakeThreshold);
11          intent.putExtra(TorchService.EXTRA_TORCH_STATE, torchOn);
12          startForegroundService(intent);
13     }
14 }

```

Listing 16: Traspaso hacia el servicio al pausar — MainActivity.java

7.4. Toggle en la interfaz

El usuario controla el modo segundo plano con un `SwitchMaterial` ubicado en la esquina superior derecha de la pantalla. Su estado se almacena en el campo `backgroundEnabled`:

```

1  SwitchMaterial switchBackground = findViewById(R.id.
2      switchBackground);
3  switchBackground.setOnCheckedChangeListener(
4      (buttonView, isChecked) -> backgroundEnabled = isChecked);

```

Listing 17: Toggle de modo segundo plano — MainActivity.java

7.5. Declaración en el Manifest

Para que Android reconozca el servicio y le permita ejecutarse en primer plano, se lo declara en `AndroidManifest.xml`:

```

1  <uses-permission android:name="android.permission.CAMERA" />

```

```

2  <uses-permission android:name="android.permission.
    FOREGROUND_SERVICE" />
3  <uses-permission android:name=
4  "android.permission.FOREGROUND_SERVICE_SPECIAL_USE" />
5  <uses-permission android:name="android.permission.
    POST_NOTIFICATIONS" />
6
7  <service
8      android:name=".TorchService"
9      android:foregroundServiceType="specialUse"
10     android:exported="false" />

```

Listing 18: Permisos y declaración del servicio — `AndroidManifest.xml`

El tipo `specialUse` es el más apropiado para este caso: la combinación de acelerómetro y control de flash no encaja en ninguna de las categorías específicas de Android 14 (cámara, ubicación, salud, etc.). `POST_NOTIFICATIONS` es requerido para mostrar la notificación persistente en Android 13 y superior. El permiso se solicita en tiempo de ejecución desde `onCreate()` de la Activity.

8. Conclusiones

Android Helios integra los dos módulos de hardware requeridos—acelerómetro y cámara/flash—de forma funcional y robusta. Los puntos clave del diseño son:

- **Detección de sacudidas en cuatro capas:** umbral de g-force + tiempo mínimo entre sacudidas + ventana de reinicio + contador mínimo. Cada capa reduce los falsos positivos independientemente.
- **TorchCallback como fuente única de verdad:** el estado de la linterna siempre refleja el estado real del hardware, sin importar qué componente lo cambió (Activity, Service, u otra app).
- **Gráfico como herramienta de calibración:** el usuario puede ver en tiempo real la magnitud de sus movimientos y ajustar el umbral hasta que la línea roja quede por debajo de sus sacudidas y por encima del ruido de fondo.
- **Ciclo de vida estricto:** el sensor nunca corre en segundo plano sin consentimiento explícito, y cuando lo hace, el sistema operativo es notificado correctamente a través del Foreground Service con su notificación persistente.